PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Smart Cloud and Server Resource Management

Student / Team Information

Team Name: Cloud-Stakers	
Team member 1 (Team Lead) Name - Suryansh Yadav Student id - 240211651 Email - 240211651@geu.ac.in	
Team member 2 Name - Varun Pratap Singh Student id - 240211710 Email - 24011710@geu.ac.in	
Team member 3 Name - Rishabh Yadav Student id - 25021069 Email - 25021069@geu.ac.in	
Team member 4 Name - Riya Saini Student id - 240221603 Email - 240221603@geu.ac.in	

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Modern cloud and server environments handle multiple applications and user requests simultaneously, with each job requiring specific amounts of CPU, memory and storage resources. Inefficient allocation of these resources can result in server overload, underutilization, increased waiting time and reduced overall system performance.

Multiple processes may also compete for limited resources, resulting in resource conflicts and potentially unsafe states. Operating System concepts such as process management, CPU scheduling, resource allocation, synchronization and deadlock handling are therefore important for efficient resource utilization.

At the same time, information related to users, servers, jobs, resource allocation and resource usage needs to be stored and managed consistently. Our goal is to develop a simulated Smart Cloud and Server Resource Management System that combines OS-based process and resource management with a structured DBMS-backed system.

The system will analyze job requirements, identify suitable servers, perform a safety check, schedule processes, allocate resources, track execution and update the database. A rule-based resource suitability mechanism will be used to select an appropriate server based on available resources and current workload.

This project provides a practical way to demonstrate how Operating Systems and DBMS concepts work together rather than treating them as two separate academic subjects.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Modern cloud platforms and server-management systems provide sophisticated mechanisms for monitoring server resources, scheduling workloads, allocating resources and balancing load across infrastructure. Production systems use large-scale distributed architectures to dynamically manage computing resources according to application requirements and system conditions.

However, these production-level platforms are highly complex and are not suitable for directly reproducing all their functionality within a semester-level academic project. At the student level, many projects focus either on Operating Systems concepts such as CPU scheduling and deadlock management or on DBMS concepts such as database design and SQL operations, while their integration is often limited.

Our proposed system focuses on creating a smaller educational simulation that explicitly combines both areas. The OS component will manage processes, scheduling, resource allocation, synchronization and deadlock handling, while the DBMS will maintain persistent information about users, servers, jobs, resource allocations and usage logs.

The project therefore provides a controlled environment for demonstrating the interaction between process/resource management and database transactions, consistency and concurrency.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

The general goal is to design and develop a simulated cloud and server resource-management system that efficiently manages multiple jobs and computing resources while demonstrating the integration of Operating Systems and Database Management Systems concepts.

Milestone 1 – Requirements and Research (Phase 1):
Identify the resource-management problem, study relevant OS concepts including process management, CPU scheduling, resource allocation, synchronization and deadlocks, and identify DBMS concepts including relational design, normalization, transactions and concurrency.

Milestone 2 – System and Database Design (Phase 1):
Define functional and non-functional requirements, design the high-level system architecture, identify database entities and relationships, and finalize the proposed technology stack and smart resource-allocation approach.

Milestone 3 – Core OS and DBMS Implementation (Phase 2):
Implement the relational database, process management, CPU scheduling using FCFS, Priority Scheduling and Round Robin, resource allocation and deallocation, synchronization and deadlock/safe-state checking.

Milestone 4 – Smart Allocation and System Integration (Phase 3):
Integrate the OS resource-management engine with the DBMS, implement rule-based smart server selection, connect job scheduling with resource allocation, and develop the application interface and dashboard.

Milestone 5 – Testing, Evaluation and Finalization (Phase 3):
Test normal allocation, insufficient-resource, server-overload, deadlock and concurrent-request scenarios; evaluate scheduling and resource-management performance; generate reports; and complete final documentation and presentation.

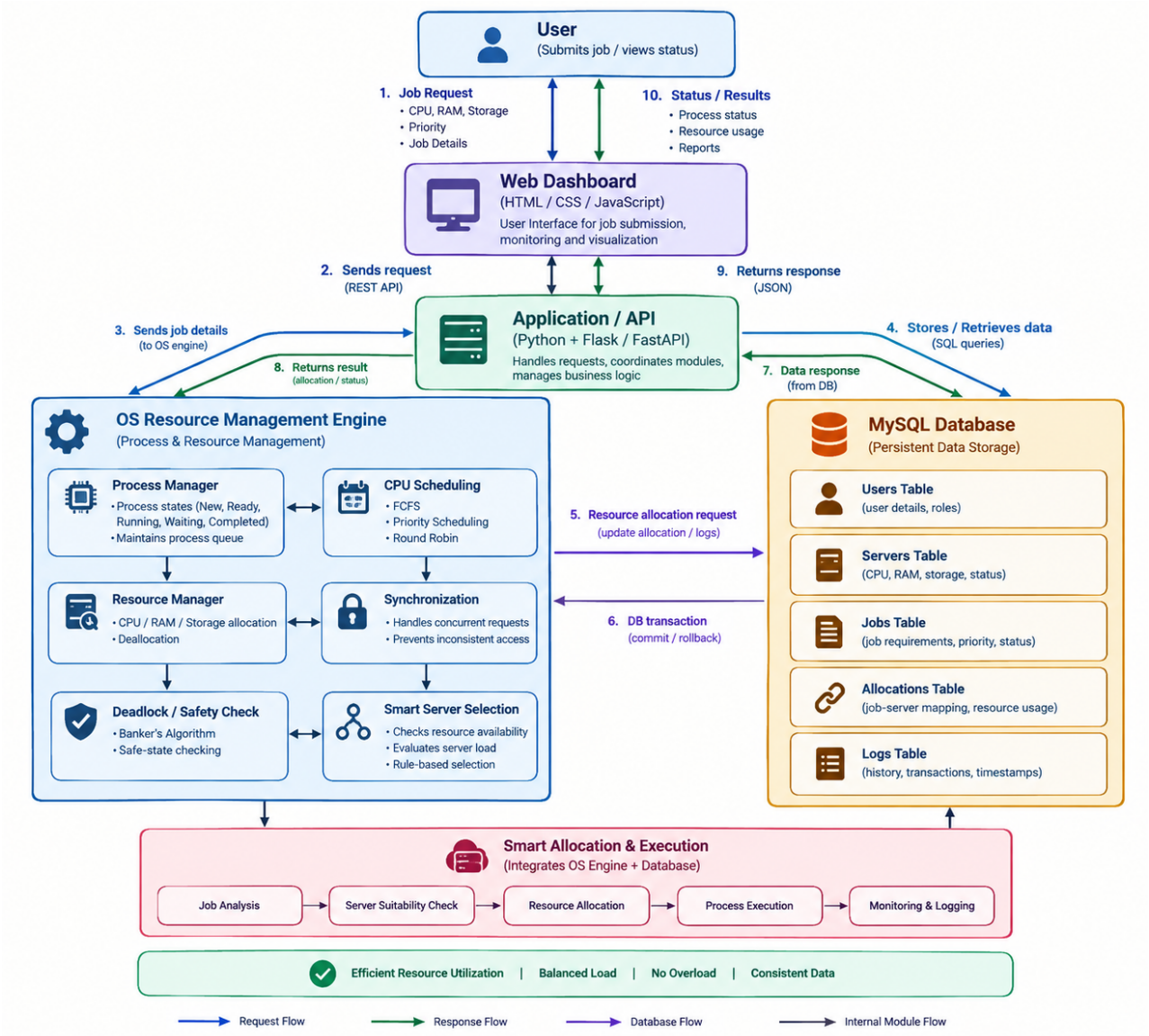
Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

We will design the system as a simulated cloud environment containing multiple logical servers. Each server will have defined CPU, RAM and storage capacities along with its current resource utilization.
Users will submit jobs containing requirements such as CPU capacity, RAM, storage and priority. The application will first store the job information in the relational database and then pass the job to the resource-management engine.
The OS component will manage the job as a process. Process states such as New, Ready, Running, Waiting and Completed will be maintained. CPU scheduling will be demonstrated using FCFS, Priority Scheduling and Round Robin algorithms, with waiting time, turnaround time and response time calculated for evaluation.
For resource allocation, the system will examine the available CPU, RAM and storage of each server. Servers that cannot satisfy the job requirements will be rejected. The remaining candidates will receive a rule-based suitability score based on available resources

and current workload. The highest-scoring suitable server will be selected.
Before final allocation, the system will perform a safe-state check using a deadlock-management mechanism such as Banker's Algorithm. Synchronization mechanisms will be used to handle simultaneous resource requests and prevent inconsistent allocation.
The DBMS will maintain users, servers, jobs, allocations and resource-usage logs. Database transactions will be used during important allocation operations so that related database updates can be committed together or rolled back when an operation fails.
The proposed technology stack consists of Python for the OS simulation and backend logic, Flask or FastAPI for application services, MySQL for the relational database, and HTML, CSS and JavaScript for the dashboard and visualization.
No AI or machine-learning component is required for the core project. The smart behavior is achieved through rule-based resource suitability and safe resource-allocation logic.

System Architecture (High Level Diagram)(2 pts)



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

(1) A working simulated cloud and server resource-management system supporting multiple jobs and logical servers;

(2) An OS process-management and CPU-scheduling module implementing FCFS, Priority Scheduling and Round Robin;

(3) A resource-management module for CPU, RAM and storage allocation and deallocation;

(4) A smart rule-based server-selection mechanism that evaluates resource availability and server workload;

(5) A deadlock/safe-state management module using Banker's Algorithm or an equivalent safety-checking mechanism;

(6) Synchronization and concurrent-request handling to maintain consistent resource allocation;

(7) A relational MySQL database containing users, servers, jobs, allocations and resource-usage logs;

(8) A dashboard displaying server utilization, process status, resource allocation and scheduling information;

(9) Testing and performance evaluation using waiting time, turnaround time, response time, resource utilization and allocation scenarios;

(10) Source code, technical documentation, final report and presentation.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words).

We assume:

(1) The system will initially simulate multiple logical servers on a single computer rather than using a commercial cloud infrastructure;

(2) CPU, RAM and storage capacities will be represented logically within the simulation;

(3) Submitted jobs will be represented as simulated processes rather than real production workloads;

(4) Server-selection scores and resource thresholds will be configurable and refined during implementation and testing;

(5) The core project does not require AI or machine-learning-based prediction and focuses on OS algorithms, DBMS concepts and rule-based resource management.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

- Z. Tong et al.e, : A novel dynamic load balancing scheduling scheme under SLA constraints in cloud computing
- A.F.S. Devaraj et al. *:* Hybridization of firefly and improved multi-objective particle swarm optimization algorithm for energy efficient load balancing in cloud computing environments
- H. Li et al. *:* Cost-aware scheduling for streaming applications in geographically distributed heterogeneous cloud
- Y. Zhang et al. *:* Adaptive deep reinforcement learning for dynamic cloud task scheduling under workload fluctuations

— End of Project Proposal —